American Sign Language translator

Author: Katarzyna Zrobek

Task:

Training a model that predicts an ASL character from a picture of a given sign

How to run/install my code:

To run my code you simply need to open the colab notebook. Then you need to mount your google drive and change the directory to the one with the “asl” folder that is enclosed with my project.
This project uses multiple libraries and it might be necessary to install them first.
Imported packages:

- `matplotlib.pyplot (v. 3.7.1)`
- `numpy (v. 1.25.2)`
- `tensorflow (v. 2.15.0)`
- `pandas (v. 2.0.3)`
- `os`
- `random (v. 3.12.4)`
- `train_test_split` from `sklearn.model_selection (v. 1.2.2)`

Dataset:

The “asl” folder includes 36 subfolders, one for each of the letters in English and one for each digit. There are 111 images of every class. Overall, there are 3996 images in the dataset. After encoding, the images become 80x80 pixels big and in grayscale.

The letters

The images of the letters come from a dataset made for an application that converts ASL to speech[1]. In the original dataset, there are 3000 images for each letter. I have decided to reduce that number for my project, which is why I have only used the images with a code number starting with 9. There were 111 images for each letter like that.

The digits

The images of the digits come from an American Sign Language Digits Dataset [2]. This dataset has 500 images for each digit. To match the number of images for letters, I have decided to also get 111 images for each digit. I have chosen all the images that had a code starting with a number in a range of 5-9 or 45-49, which gave me 111 images for each digit.

Training:

I have used convolutional neural network architecture for this task. It was built using TenserFlow's Keras API.
After a few trials with different parameters (changing LR, amount of epochs and using momentum / Adam optimizer) I have discovered that a model trained with LR = 0.001, epochs = 25 and using Adam optimizer gives the optimal results, with loss = 0.09 and accuracy = 0.98. While a bigger amount of epochs slightly increases the accuracy and decreases the loss, the changes are negligible in comparison how much longer the task takes time. Moreover, I didn't want to overfit my model.
And log loss & accuracy over the epochs for the best model:

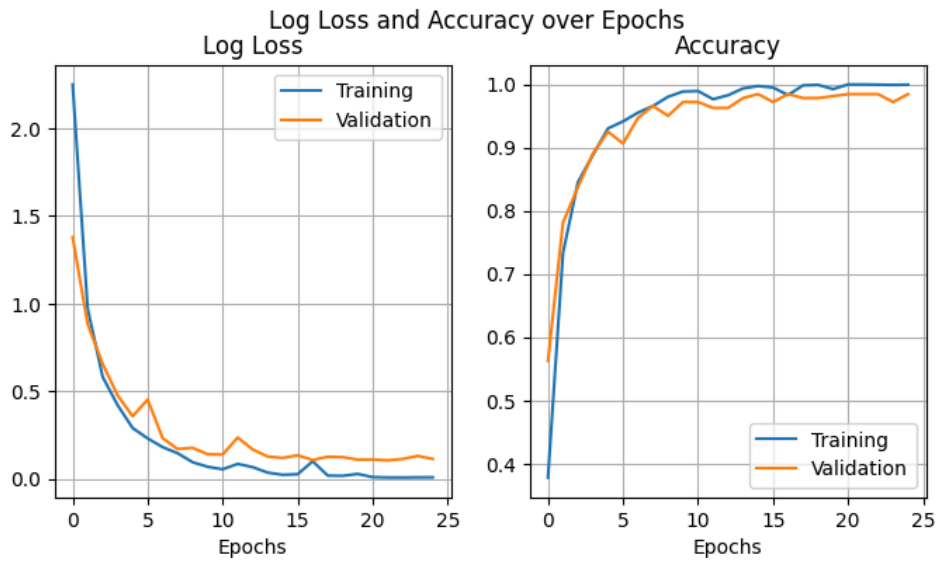

Table showing the results of some some trials I ran:

Parameters	Loss	Accuracy
LR=0.0006, epochs=20, momentum=0.0	2.07	0.49
LR=0.0006, epochs=20, momentum=0.9	0.2	0.95
LR=0.0006, epochs=20, optimizer='Adam'	0.13	0.977
LR=0.001, epochs=20, optimizer='Adam'	0.13	0.980
LR=0.05, epochs=20, optimizer='Adam'	0.11	0.972
LR=0.08, epochs=20, optimizer='Adam'	0.15	0.977
LR=0.005, epochs=20, optimizer='Adam'	0.14	0.977
LR=0.001, epochs=15, optimizer='Adam'	0.1	0.977
LR=0.001, epochs=25, optimizer='Adam'	0.09	0.982
LR=0.001, epochs=30, optimizer='Adam'	0.08	0.981
LR=0.001, epochs=35, optimizer='Adam'	0.07	0.982

Next, I have run the model 5 times, with different split of the dataset for training / test:

Accuracy	test_size = 0.2	test_size = 0.3
mean	0.977	0.976
standard deviation	0.005	0.003
best	0.981	0.98

It seems like the model gives consistent results and different dataset splits produce only slightly different results.

Finally, I have made a table that shows which shows the accuracy for given characters. For example, we can see that the model struggled most with characters “6” and “7”.

index	Label	Total data	Training data	Test data	Correct predictions	Accuracy
0	0	111	89	22	22	100.0
1	1	111	89	22	22	100.0
2	2	111	89	22	22	100.0
3	3	111	89	22	22	100.0
4	4	111	89	22	20	90.9
5	5	111	89	22	21	95.5
6	6	111	89	22	16	72.7
7	7	111	89	22	19	86.4
8	8	111	89	22	22	100.0
9	9	111	89	22	21	95.5
10	A	111	89	22	21	95.5
11	B	111	89	22	22	100.0
12	C	111	89	22	22	100.0
13	D	111	88	23	23	100.0
14	E	111	88	23	23	100.0
15	F	111	88	23	23	100.0
16	G	111	88	23	23	100.0
17	H	111	88	23	21	91.3
18	I	111	89	22	22	100.0
19	J	111	88	23	22	95.7
20	K	111	89	22	22	100.0
21	L	111	89	22	21	95.5
22	M	111	88	23	23	100.0
23	N	111	89	22	22	100.0
24	O	111	89	22	21	95.5
25	P	111	89	22	22	100.0
26	Q	111	89	22	22	100.0
27	R	111	89	22	22	100.0
28	S	111	89	22	22	100.0
29	T	111	89	22	22	100.0
30	U	111	89	22	22	100.0
31	V	111	88	23	23	100.0
32	W	111	89	22	22	100.0

Bibliography:

1. Akash Nagaraj. (2018). ASL Alphabet [Data set]. Kaggle. https://doi.org/10.34740/KAGGLE/DSV/29550
2. A New 2D Static Hand Gesture Colour Image Dataset for ASL Gestures - A.L.C. Barczak, N.H. Reyes, M. Abastillas, A. Piccio and T. Susnjak; https://www.kaggle.com/datasets/rayeed045/american-sign-language-digit-dataset